Machine Learning in Python

Lecture 2: From the perceptron to a neural network. Data as tensors

Przemysław Rola

Department of Mathematics, Krakow University of Economics

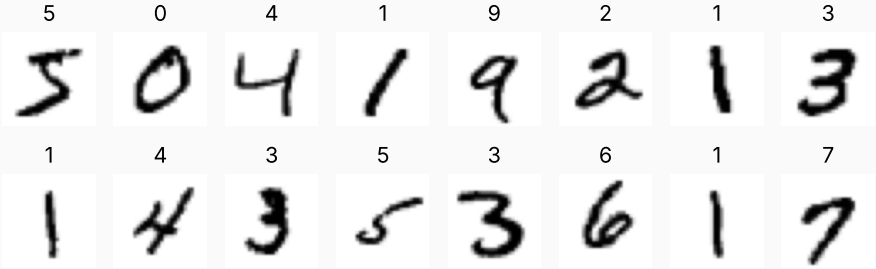
Plan

1. The task: handwritten digits
2. From the perceptron to a multilayer network
3. The first network in Keras
4. Data as tensors

Based on F. Chollet, M. Watson, *Deep Learning with Python*, 3rd ed., Ch. 2.

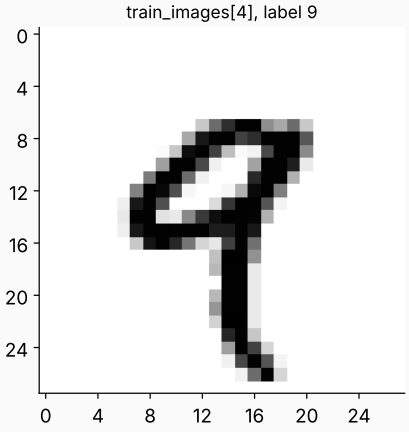
The task: handwritten digits

MNIST: the "hello world" of deep learning



- Grayscale images 28×28 pixels, pixel values 0 (white) to 255 (black).
- 10 classes: digits $0, \ldots, 9$. 60 000 training images, 10 000 test images.
- Assembled by LeCun, Cortes and Burges from data of the US National Institute of Standards and Technology (NIST).

What does the computer see?



- An image is a **table of numbers**: 28 rows, 28 columns.
- For the networks of today we flatten it into one long list of $28 \cdot 28 = 784$ numbers $x_1, \ldots, x_{784}$.
- Then each image is just a point with 784 coordinates — exactly the kind of input a perceptron takes.
- Question for later: what information do we lose by flattening?

From the perceptron to a multilayer network

Recall: one perceptron, one decision

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b, \qquad \hat{y} = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

- One perceptron answers one yes/no question with one straight line.
- For 10 digits we need 10 answers, and the classes are not separated by straight lines.

Three changes lead to a neural network:

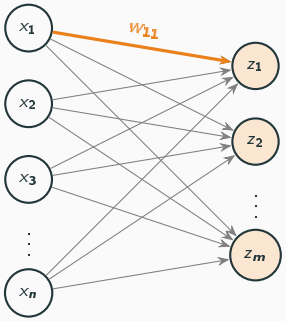
1. many units side by side $\rightarrow$ a **layer**;
2. several layers one after another, with a **nonlinear activation** in between;
3. a smooth output (probabilities) and a smooth **loss**, so that we can train with **gradient descent**.

A layer = many units computing in parallel

Unit j of the layer has its own weights $w_{1j}, \ldots, w_{nj}$ and bias b_j:

$$z_j = w_{1j}x_1 + w_{2j}x_2 + \cdots + w_{nj}x_n + b_j.$$

- A layer with m units turns n input numbers into m new numbers.
- All weights form a table W with n rows and m columns; the biases form a list b of length m.
- This is called a **dense** (fully connected) layer: every input is connected to every unit.



n inputs $\rightarrow$ m outputs

Why do we need an activation between layers?

Stacking two layers *without* activation gives nothing new. One input, one unit per layer:

$$\text{layer 1: } h = a\,x + c, \qquad \text{layer 2: } y = d\,h + e = d(ax + c) + e = \underbrace{(da)}_{\text{slope}}\,x + \underbrace{(dc + e)}_{\text{intercept}}.$$

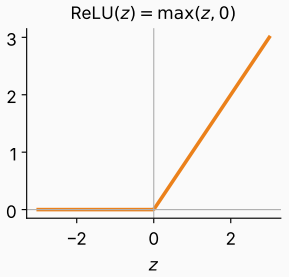
Again a single linear function. The same holds for any number of linear layers and inputs.

Fix: a nonlinear activation

Apply a simple nonlinear function to every output of a hidden layer. Today's default:

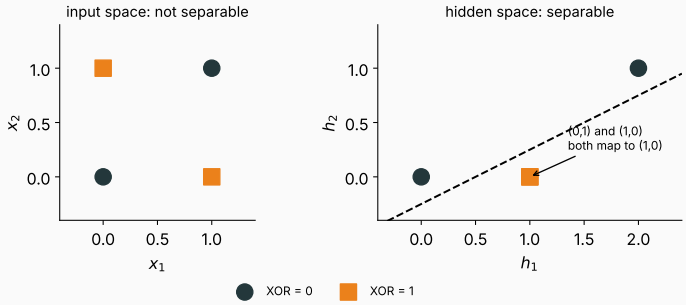
$$\mathrm{ReLU}(z) = \max(z, 0)\,:$$

negative values become 0, positive values stay unchanged.



Example: XOR with one hidden ReLU layer

Hidden layer: $h_1 = \mathrm{ReLU}(x_1 + x_2), \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1).$



In the new coordinates (h_1, h_2) one straight line separates the classes; the output unit $y = h_1 - 2h_2$ gives exactly XOR (check: $0, 1, 1, 0$). The hidden layer learns a new representation of the data.

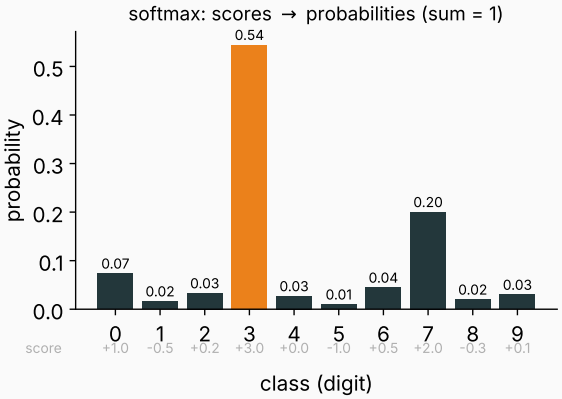
(Goodfellow et al. 2016, Sec. 6.1)

The output layer: probabilities with softmax

- The last layer has 10 units, one per digit: scores $z_0, \ldots, z_9$.
- **Softmax** turns scores into probabilities:

$$p_k = \frac{e^{z_k}}{e^{z_0} + e^{z_1} + \cdots + e^{z_9}}.$$

- All $p_k > 0$ and $p_0 + \cdots + p_9 = 1$; a larger score gives a larger probability.
- Prediction: the class with the largest p_k.

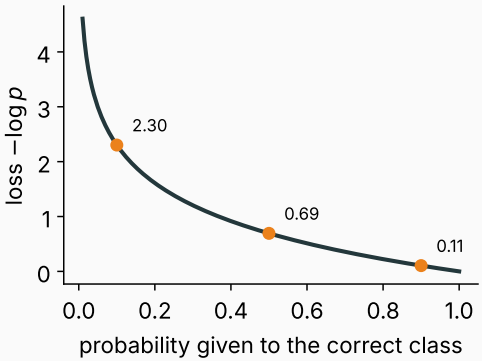


How wrong is the network? The loss function

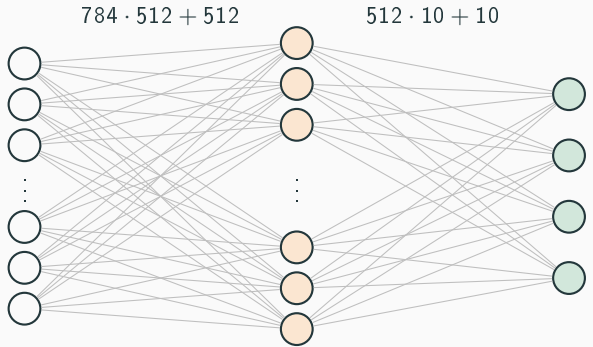
For one image with correct class k:

$$\text{loss} = -\log p_k \qquad \text{(cross-entropy)}$$

- p_k close to 1 (confident and right) $\Rightarrow$ loss close to 0.
- p_k close to 0 (confident and wrong) $\Rightarrow$ very large loss.
- Training loss = average over the images.
- Unlike "number of mistakes", this loss changes smoothly with the weights $\Rightarrow$ we can use derivatives (Lecture 3).



Our first network: a multilayer perceptron (MLP)



Number of parameters (weights and biases): $401\,920 + 5\,130 = 407\,050$. All are learned from data.

The first network in Keras

Step 1: load the data

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images.shape     # (60000, 28, 28)
train_labels[:5]       # array([5, 0, 4, 1, 9], dtype=uint8)
test_images.shape      # (10000, 28, 28)
```

- **Training set**: the model learns from it.
- **Test set**: kept aside, used only at the end to check how well the model works on new images.

Step 2: define the network

```
import keras
from keras import layers

model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
```

- `Sequential`: layers applied one after another.
- `Dense(512, activation="relu")`: hidden layer, 512 units, ReLU.
- `Dense(10, activation="softmax")`: 10 probabilities, one per digit.
- The number of inputs (784) is inferred from the data on first use.

Step 3: compile — choose how to learn

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

loss what to minimise: cross-entropy. "sparse" = labels are given as integers 0..9.

optimizer how to change the weights: Adam, a variant of gradient descent (Lecture 3).

metrics what to report: accuracy = fraction of correctly classified images. Not used for training.

Step 4: prepare the data

```
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

- `reshape`: each 28×28 image becomes a row of 784 numbers $\Rightarrow$ shape `(60000, 784)`.
- `/ 255`: pixel values from 0..255 to 0..1. Neural networks train much better on small numbers of similar size.
- Integers `uint8` $\rightarrow$ floating-point numbers `float32`.

Step 5: train (“fit”)

```
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

```
Epoch 1/5   469/469 - loss: 0.26 - accuracy: 0.92
...
Epoch 5/5   469/469 - loss: 0.04 - accuracy: 0.99
```

- **Batch**: the weights are updated after every 128 images.
- One **epoch** = one pass over all 60 000 images = $\lceil 60000/128 \rceil = 469$ updates.
- 5 epochs = 2 345 updates of all 407 050 parameters.
- The loss goes down and the accuracy on the training data goes up.

Numbers vary slightly between runs (random initial weights).

Step 6: predict and evaluate

```
predictions = model.predict(test_images[:10])
predictions[0].round(3)   # 10 probabilities, the largest at index 7
predictions[0].argmax()   # 7    <- predicted digit
test_labels[0]            # 7    <- true digit

test_loss, test_acc = model.evaluate(test_images, test_labels)
print(test_acc)           # about 0.98
```

- Test accuracy $\approx$ 97.8%, training accuracy $\approx$ 98.9% (values reported by Chollet & Watson).
- The model is slightly better on data it has seen: **overfitting**. More in a later lecture.

Data as tensors

Tensors: containers for numbers

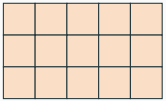
A **tensor** is a multidimensional array of numbers. (TensorFlow is named after tensors, not the other way round.)



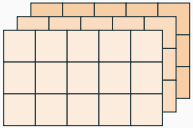
scalar
rank 0
shape `()`



vector
rank 1
shape `(5,)`



matrix
rank 2
shape `(3, 5)`



rank-3 tensor
shape `(3, 3, 5)`

- **rank** = number of axes = `x.ndim`. (Not the same as the rank of a matrix in linear algebra!)
- **shape** = length along each axis = `x.shape`. **dtype** = type of the numbers, e.g. `uint8`, `float32`.

Tensors in NumPy

```
import numpy as np
x = np.array(12)
x.ndim                  # 0

x = np.array([12, 3, 6, 14, 7])
x.ndim, x.shape         # 1, (5,)

x = np.array([[5, 78, 2, 34, 0],
              [6, 79, 3, 35, 1],
              [7, 80, 4, 36, 2]])
x.ndim, x.shape         # 2, (3, 5)
```

```
train_images.ndim       # 3
train_images.shape      # (60000, 28, 28)
train_images.dtype      # uint8
```

- MNIST training images: 60 000 matrices of size 28×28 of integers 0..255.
- Careful with words: `[12, 3, 6, 14, 7]` is a "5-dimensional vector" (5 entries) but a rank-1 tensor (1 axis).

Selecting parts of a tensor: slicing

```
train_images[4]                  # one image, shape (28, 28)
train_images[10:100]             # images 10..99, shape (90, 28, 28)
train_images[10:100, :, :]       # the same, all rows and columns
train_images[:, 14:, 14:]        # bottom-right 14x14 corner of every image
train_images[:, 7:-7, 7:-7]      # central 14x14 patch of every image
```

- `a:b` means indices $a, a+1, \ldots, b-1$; a negative index counts from the end.
- One index per axis; `:` means "everything along this axis".
- In the lab: cut off a border of the images and see what the network thinks of it.

The first axis: samples and batches

```
batch = train_images[:128]           # first batch
batch = train_images[128:256]        # second batch
n = 3
batch = train_images[128 * n : 128 * (n + 1)]   # batch number n
```

- By convention axis 0 is the **samples axis** (also: batch axis).
- Models do not look at all data at once, but at **batches** of samples.

Real-world data as tensors

Data	Shape	Example
Tables (vector data)	`(samples, features)`	100 000 clients, 3 features (age, income, debt)
Time series	`(samples, timesteps, features)`	250 days, 390 minutes, 3 prices (current, high, low)
Images	`(samples, height, width, channels)`	128 colour photos, 256×256, 3 channels (RGB)
Video	`(samples, frames, h, w, channels)`	4 clips, 240 frames, 144×256, 3 channels

- A grayscale image has 1 channel: MNIST could be stored as `(60000, 28, 28, 1)`.
- Order of axes for images: *channels-last* `(samples, h, w, c)` — Keras/TensorFlow default; *channels-first* `(samples, c, h, w)` — PyTorch default.

Examples after Chollet & Watson, Sec. 2.2.

Summary

- A **dense layer** = many perceptron-like units; a **network** = layers stacked, with **ReLU** in between.
- Without a nonlinear activation, many layers are no better than one.
- Output: **softmax** probabilities; training objective: **cross-entropy loss**.
- Keras workflow: `Sequential` → `compile` → `fit` → `evaluate` / `predict`.
- Data are **tensors**: rank, shape, dtype; axis 0 = samples; we feed the model in batches.

Next: what happens inside `fit`? Tensor operations and gradient descent.

References

- F. Chollet, M. Watson (2025). *Deep Learning with Python*, 3rd ed. Manning. Ch. 2. `https://deeplearningwithpython.io`
- I. Goodfellow, Y. Bengio, A. Courville (2016). *Deep Learning*. MIT Press. Sec. 6.1 (XOR example). `https://www.deeplearningbook.org`
- Y. LeCun, L. Bottou, Y. Bengio, P. Haffner (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11), 2278–2324.